

for areas with high levels of functional safety requirements (see [Figure 4-1](#)).

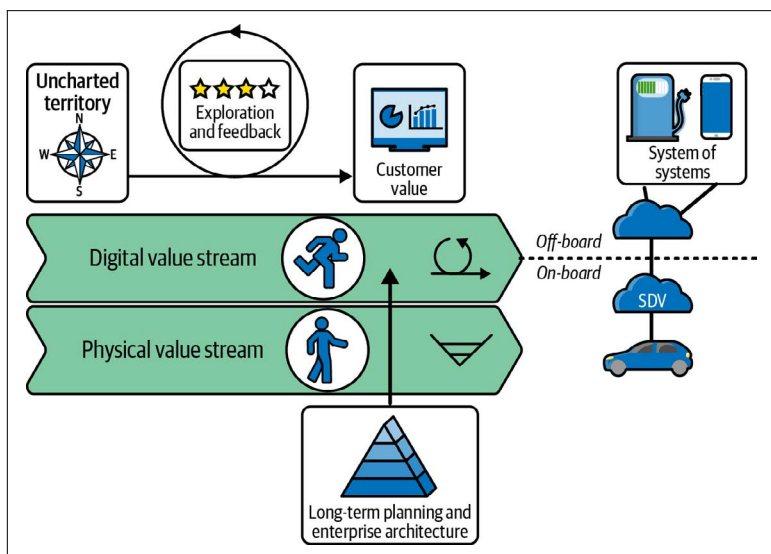


Figure 4-1. High-level value stream perspective for the SDV

The digital value stream must be able to address uncharted territory, deal with vague requirements and constantly changing ideas, and be able to quickly react to customer feedback. Building an explorative and feedback-based approach to the digital value stream is key. The technical delivery pipelines of the digital value stream will be outputting artifacts that are deployed either in the cloud or on-board. The on-board artifacts will use SDV, containerization, OTA, and vehicle APIs. Mechanisms for measuring customer success can be natively built into this technology stack, similar to the customer journey analysis tools familiar from internet applications.

The physical value stream must be aligned much more closely with long-term planning and the enterprise architecture. The methods applied here will adhere to established best practices for functional safety-related features (e.g., the well-established V-model of software development, which has formal verification and validation mechanisms built in). The technical outputs of the physical value stream will include a combination of hardware and software, such as embedded software and ECUs, as well as mechatronic system components. Implementing mechanisms for measuring customer success in the physical value stream will be more difficult.

Establishing an effective VSM strategy that enables OEMs to work at different speeds in different areas will be a key success factor in the future.

Divide and Conquer

The beauty of SOA and the API-centric way of working that we introduced earlier is that these approaches give us the capability to create integration not only on the technical level but also on the organizational level. To support value streams that are moving at different speeds, a loose coupling is required both on the technical as well as on the organizational level. And this is exactly what can be achieved with APIs and hardware abstraction.

Figure 4-2 shows an example of loose coupling. At the top, we have technical artifacts created as output of the digital value stream. First, prototypes are created and implemented against the vehicle API—e.g., using the **digital.auto playground** for rapid online prototyping of SDV features. These prototypes can be used to get very early feedback from key stakeholders, including customers and management. Next, the early SDV prototypes are refined and brought closer to real applications. In the early development phases, these SDV applications can utilize vehicle simulations behind the APIs to create realistic test environments. Once real vehicle hardware is available, this hardware can replace the vehicle simulation. Since the APIs are not changing (at least in an ideal world) and SDV application is implemented against the API, this shift from a vehicle simulation to real vehicle hardware should be seamless from the point of view of the SDV application. This means that the API becomes the mechanism that ensures loose coupling not only between the technical artifacts but also between the different organizations involved. The teams developing the digital value stream are doing so against the APIs representing the real hardware underneath (hence “hardware abstraction”), and by using simulators, they can move at their own speed, independent of the availability of the real hardware. This approach takes the well-established concepts of hardware-in-the-loop (HIL) and software-in-the-loop (SIL) to another level.

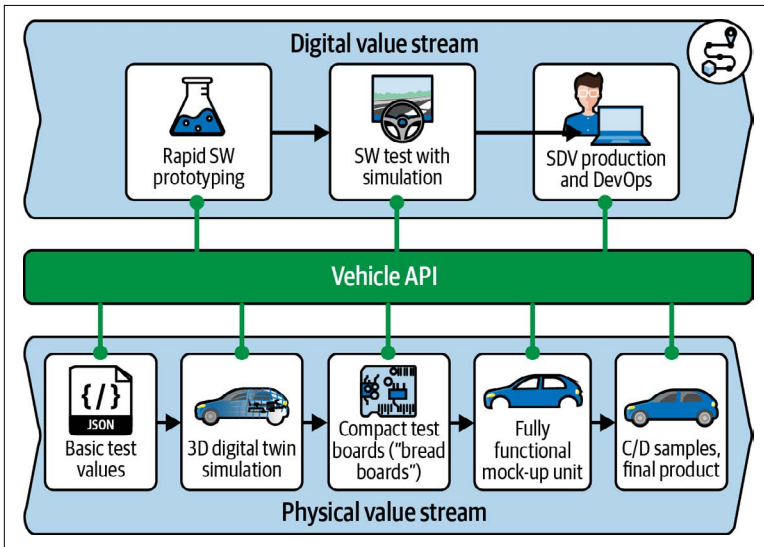


Figure 4-2. Value streams with technical artifacts moving at different speeds

Being able to split up a complex body of work using a divide and conquer strategy is a huge benefit, reducing complexity to a manageable level. Notice that the APIs are likely to take on the role of a “master clock,” helping to synchronize work between the different teams in the different value streams.

Enterprise Perspective

OEMs have traditionally addressed the complexity they face with enterprise architecture management (EAM) and Model-Based Systems Engineering (MBSE), shown in [Figure 4-3](#). EAM helps manage the dependencies between the systems-of-systems perspective (e.g., the vehicle in the context of its environment), the system perspective (the vehicle itself), as well as the subsystems, including key components and features. Of course, all of this must be seen in the context of many different vehicle variants and vehicle types. Finally, managing the reuse of vehicle platforms is key, including hardware platforms, E/E platforms, and software platforms. MBSE plays an increasingly important role in the detailed design of many system components and their interdependencies.

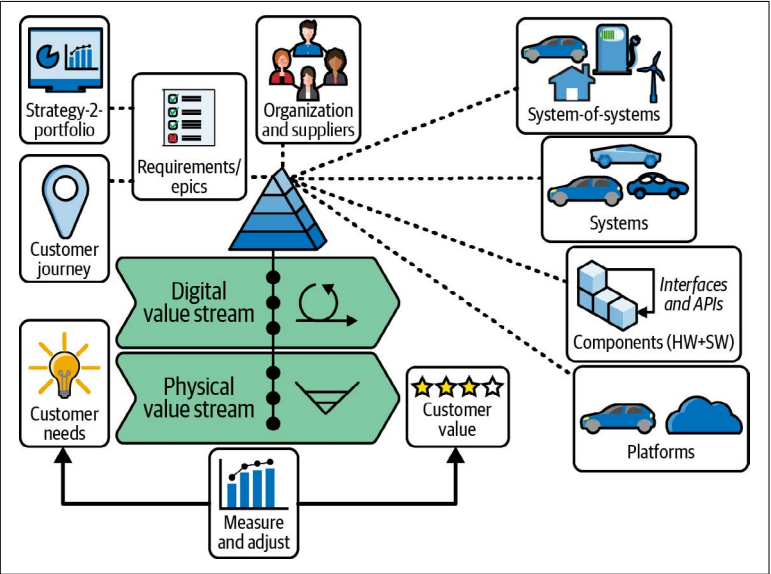


Figure 4-3. The enterprise perspective

However, coming back to the “clash of two worlds” discussion, it is important to notice that these kinds of tools and methods are often seen as very controversial in the software world, which has spent the last 20 years adopting an Agile culture. This becomes clear in [Table 4-1](#), which compares how the Agile values defined in the famous [manifesto for Agile software development](#) map to model-centric and code-centric approaches to automotive development.

Table 4-1. Model-centric versus code-centric development

Values from the Agile Manifesto	Model-centric (EAM/MBSE)	Code-centric/SDV
Individuals and interactions over processes and tools	Tools and processes required, especially for hardware and software with ASIL requirements	Well suited to Agile processes for QM features
Working software over comprehensive documentation	Can be achieved via code generated from models (but not a trivial task)	Well supported
Customer collaboration over contract negotiation	Model as contract; early customer validation requires virtual exploration	APIs as contracts, but features developed in close alignment with customers
Responding to change over following a plan	Long-term planning required for hardware and ASIL software	Supported by Agile approach

As we have discussed before, the ability to work with different value streams that embody different approaches and methods is the key to success. On the enterprise-level, methods and mechanisms must be established to keep the different value streams in sync, supported by a loose-coupling approach on the organizational level. Again, APIs can play a key role here, for example, by providing a loose coupling between the Agile/code-centric and the model-centric/MBSE perspective.